

Documentación Técnica del Buscador

Índice

- [1. Introducción](#)
 - [2. Arquitectura del Sistema](#)
 - [3. Componentes Principales](#)
 - [4. Funcionamiento Detallado](#)
 - [5. Configuración](#)
 - [6. Integración](#)
 - [7. Internacionalización](#)
-

Introducción

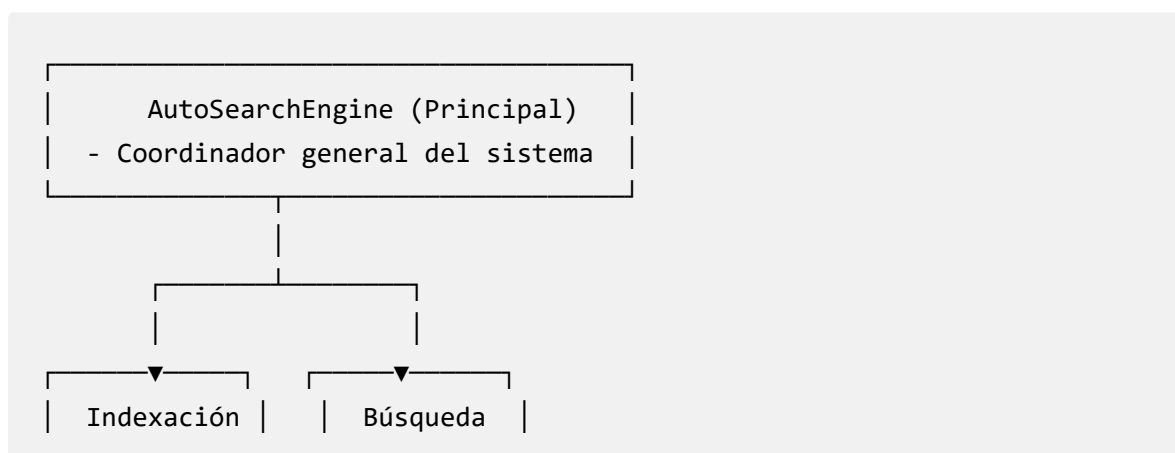
El buscador implementado en este sitio web personal es un sistema de búsqueda de contenido en el lado del cliente (client-side) que indexa todas las páginas del sitio y permite realizar búsquedas en tiempo real.

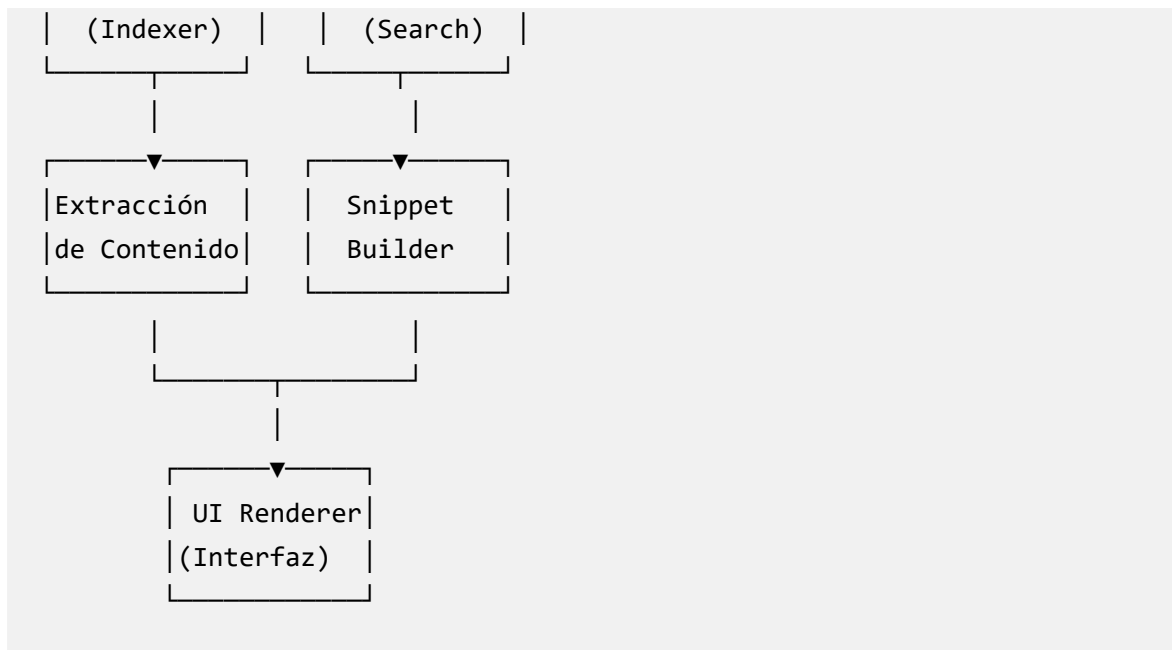
Características Principales

- **Búsqueda en tiempo real:** Los resultados se muestran mientras el usuario escribe
 - **Indexación automática:** Todas las páginas se indexan al cargar el sitio
 - **Soporte multiidioma:** Compatible con español e inglés
 - **Búsqueda ponderada:** Diferentes secciones tienen diferentes pesos de relevancia
 - **Snippets contextuales:** Muestra fragmentos de texto relevantes con el término resaltado
 - **Interfaz accesible:** Cumple con estándares de accesibilidad web
-

Arquitectura del Sistema

El buscador está diseñado con una arquitectura modular orientada a objetos, dividida en las siguientes capas:





Flujo de Trabajo

1. **Inicialización:** Al cargar la página, se inicializa i18next y el motor de búsqueda
2. **Indexación:** Se descargan y procesan todas las páginas definidas en `pagesToIndex`
3. **Búsqueda:** Cuando el usuario escribe, se busca en el índice
4. **Renderizado:** Los resultados se muestran en un overlay modal

Componentes Principales

1. AutoSearchEngine

Responsabilidad: Clase principal que coordina todos los componentes del sistema.

Métodos públicos:

- `indexPages(pages, forceReindex)`: Indexa las páginas del sitio
- `search(query)`: Realiza una búsqueda y retorna resultados
- `displayResults(results, query)`: Muestra los resultados en la UI
- `clearResults()`: Limpia los resultados mostrados
- `initializeUI()`: Configura los eventos de la interfaz

Ejemplo de uso:

```
const engine = new AutoSearchEngine();
engine.initializeUI();
await engine.indexPages(pagesToIndex);
const results = engine.search("programación");
```

2. PageIndexer

Responsabilidad: Gestiona la descarga y procesamiento de páginas para crear el índice.

Proceso de indexación:

1. Descarga el HTML de cada página mediante `fetch()`
2. Espera a que `i18next` esté inicializado para traducir contenido
3. Parsea el HTML y elimina elementos excluidos (nav, footer, scripts, etc.)
4. Traduce elementos con `data-i18n` al idioma actual
5. Extrae el contenido principal, título y descripción
6. Retorna un objeto con los datos indexados

Estructura del objeto indexado:

```
{
  url: "about.html",
  title: "Sobre mí",
  description: "Información personal y profesional...",
  content: "Me llamo Daniela Rodríguez Cepero...",
  nameKey: "nav.sobre-mi",
  metaKey: "meta.about",
  sections: {
    references: "Dra. C. Aymée de los Ángeles..."
  }
}
```

3. HTMLParser

Responsabilidad: Procesa el HTML descargado y lo prepara para indexación.

Funciones:

- Crea un contenedor DOM temporal para manipular el HTML
- Elimina elementos excluidos (navegación, footer, scripts, SVG, etc.)
- Aplica traducciones mediante `ContentTranslator`

4. ContentExtractor

Responsabilidad: Extrae el contenido relevante de las páginas.

Proceso:

1. Localiza el elemento `<main>` o `<body>`
2. Elimina secciones excluidas (redes sociales)
3. Convierte encabezados (`h1-h6`) en texto plano con separador ":"
4. Extrae secciones especiales como referencias
5. Limpia espacios en blanco redundantes

5. ContentTranslator

Responsabilidad: Traduce elementos con atributos `data-i18n`.

Funcionamiento:

- Busca todos los elementos con `[data-i18n]`
- Obtiene la traducción del idioma actual mediante `i18next`
- Reemplaza el contenido o placeholder según el tipo de elemento

6. SearchEngine

Responsabilidad: Realiza búsquedas en el índice y calcula relevancia.

Algoritmo de relevancia:

```
relevancia = (coincidencias_en_título × 10) +  
             (coincidencias_en_descripción × 5) +  
             (coincidencias_en_contenido × 1) +  
             (coincidencias_en_secciones × 3)
```

Características:

- Búsqueda insensible a mayúsculas/minúsculas
- Los resultados se ordenan por relevancia descendente
- Solo retorna páginas con relevancia > 0

7. SnippetBuilder

Responsabilidad: Genera fragmentos de texto contextuales con el término de búsqueda.

Proceso:

1. Selecciona el mejor texto donde aparece el término
2. Extrae el contexto alrededor del término (oración completa si es posible)
3. Elimina prefijos redundantes
4. Añade "..." si el fragmento está truncado
5. Resalta el término buscado con `<mark>`

Configuración de snippet:

- Longitud máxima: 300 caracteres
- Contexto antes: 100 caracteres
- Contexto después: 150 caracteres

8. TextProcessor

Responsabilidad: Procesamiento avanzado de texto para snippets.

Funciones principales:

- `extractContext()`: Extrae contexto alrededor del término
- `findSentenceStart()` / `findSentenceEnd()`: Encuentra límites de oraciones
- `adjustToWordBoundary()`: Ajusta posiciones a límites de palabras
- `removeRedundantPrefixes()`: Elimina prefijos repetitivos
- `highlight()`: Resalta términos con `<mark>`
- `escapeRegex()`: Escapa caracteres especiales para regex

9. UIRenderer

Responsabilidad: Gestiona la presentación de resultados en la interfaz.

Funciones:

- Muestra/oculta el overlay de resultados
- Renderiza la lista de resultados
- Muestra mensaje cuando no hay resultados
- Bloquea scroll del body cuando el overlay está visible

10. TemplateRenderer

Responsabilidad: Genera el HTML para los resultados.

Templates:

- `renderNoResults()`: Mensaje de sin resultados
- `renderResults()`: Lista completa de resultados
- `renderResultItem()`: Item individual de resultado
- Escapa HTML para prevenir XSS

11. SearchEventManager

Responsabilidad: Gestiona eventos de la interfaz de búsqueda.

Eventos configurados:

- **Submit del formulario:** Ejecuta búsqueda
- **Input:** Búsqueda con debounce (300ms)
- **Click en botón cerrar:** Cierra overlay
- **Click en overlay:** Cierra si se hace click fuera
- **Tecla Escape:** Cierra overlay

Debounce: Evita búsquedas excesivas esperando 300ms después del último carácter escrito.

Funcionamiento Detallado

Proceso de Inicialización

```

// 1. Se espera a que el DOM esté listo
document.addEventListener('DOMContentLoaded', async () => {

    // 2. Se espera a que i18next esté inicializado
    if (typeof i18next !== 'undefined') {
        await new Promise(resolve => {
            if (i18next.isInitialized) {
                resolve();
            } else {
                i18next.on('initialized', resolve);
            }
        });
    }

    // 3. Se inicializa el motor de búsqueda
    await initializeSearchEngine();
});

async function initializeSearchEngine(customConfig = {}, pages = pagesToIn
    // Crear instancia del motor
    autoSearchEngine = new AutoSearchEngine(customConfig);

    // Configurar eventos de UI
    autoSearchEngine.initializeUI();

    // Indexar páginas
    await autoSearchEngine.indexPages(pages);

    return autoSearchEngine;
}

```

Proceso de Búsqueda

Usuario escribe → Debounce (300ms) → search(query) → Cálculo de relevancia
 Ordenar resultados → Generar snippets → Renderizar en UI → Mostrar overlay

Detalle paso a paso:

1. Usuario escribe en el campo de búsqueda

```

input.addEventListener('input', (e) => {
    this.debounceSearch(e.target.value);
});

```

```
});
```

2. Debounce espera 300ms

```
clearTimeout(this.debounceTimer);  
this.debounceTimer = setTimeout(() => {  
  this.executeSearch(query);  
}, 300);
```

3. Se ejecuta la búsqueda

```
const results = this.engine.search(query);
```

4. Se calcula relevancia para cada página

```
for (const page of this.index) {  
  const relevance = this.calculateRelevance(page, term);  
  if (relevance > 0) {  
    matches.push({ page, relevance });  
  }  
}
```

5. Se ordenan por relevancia

```
matches.sort((a, b) => b.relevance - a.relevance);
```

6. Se generan snippets

```
results.map(result => ({  
  ...result,  
  snippet: this.snippetBuilder.buildSnippet(result.page, query)  
}));
```

7. Se renderizan en la UI

```
container.innerHTML = this.templateRenderer.renderResults(results, qu  
overlay.style.display = 'block';
```

Cuando el usuario cambia el idioma, el buscador se re-indexa automáticamente:

```
next.on('languageChanged', async (lng) => {
  if (autoSearchEngine && typeof pagesToIndex !== 'undefined') {
    console.log(`🔄 Re-indexando páginas en ${lng}...`);
    await new Promise(resolve => setTimeout(resolve, 100));
    await autoSearchEngine.indexPages(pagesToIndex, true);
  }
});
```

Esto asegura que las búsquedas se realicen en el contenido traducido.

Configuración

Configuración por Defecto

```
const DEFAULT_SEARCH_CONFIG = {
  // Longitud mínima de consulta para activar búsqueda
  minQueryLength: 2,

  // Retardo en milisegundos para debounce
  debounceDelay: 300,

  // Pesos de relevancia para diferentes secciones
  relevanceWeights: {
    title: 10,      // Coincidencias en título
    description: 5, // Coincidencias en descripción
    content: 1,     // Coincidencias en contenido general
    sections: 3     // Coincidencias en secciones especiales
  },

  // Configuración de snippets
  snippet: {
    maxLength: 300, // Longitud máxima del snippet
    contextBefore: 100, // Caracteres antes del término
    contextAfter: 150 // Caracteres después del término
  },

  // Selectores CSS de elementos
  selectors: {
    resultsContainer: '#search-results',
    overlay: '#search-results-overlay',
    form: '#search-form',
```



```

    input: '#search-input',
    closeButton: '#close-search'
  },

  // Elementos HTML a excluir de la indexación
  excludedElements: ['script', 'style', 'nav', 'header', 'footer', 'svg'],

  // Clases CSS a excluir
  excludedClasses: ['.social-links', '.social-link'],

  // Secciones a excluir por patrón de texto
  excludedSections: {
    social: [/redes\s+social/i, /social\s+network/i]
  },

  // Prefijos redundantes a eliminar de snippets
  redundantPrefixes: [
    /^sobre mí\s+quién soy\s+/i,
    /^sobre mí\s+/i,
    /^quién soy\s+/i,
    /^referencias\s+personas que pueden avalar.+s*:\s*/i,
    /^about me\s+who i am\s+/i,
    /^about me\s+/i,
    /^who i am\s+/i,
    /^references\s+people who can vouch.+s*:\s*/i
  ],

  // Caracteres que delimitan palabras
  wordBoundaryChars: [' ', '.', ':', '\n'],

  // Claves de traducción para mensajes
  translationKeys: {
    noResults: 'search.no-results',
    results: 'search.results',
    result: 'search.result',
    resultsPlural: 'search.results-plural'
  }
};

```

Personalización

Para personalizar la configuración, pasa un objeto al inicializar:

```
const customConfig = {
  minQueryLength: 3, // Requiere 3 caracteres mínimo
  debounceDelay: 500, // Espera 500ms
  relevanceWeights: {
    title: 15, // Mayor peso al título
    description: 8,
    content: 2,
    sections: 5
  }
};

await initializeSearchEngine(customConfig, pagesToIndex);
```

Integración

Requisitos HTML

Cada página debe incluir:

1. Formulario de búsqueda en el header:

```
<form id="search-form" role="search" class="header-search">
  <input
    type="search"
    id="search-input"
    placeholder="Buscar en el sitio..."
    aria-label="Buscar en el sitio">
  <button type="submit" class="search-btn" aria-label="Buscar">
    <!-- Icono de búsqueda -->
  </button>
</form>
```

2. Overlay para resultados:

```
<aside id="search-results-overlay" class="search-overlay"
  role="dialog" aria-modal="true"
  aria-label="Resultados de búsqueda"
  style="display: none;">
  <section class="search-results-container">
    <button id="close-search" class="close-search-btn"
      aria-label="Cerrar búsqueda">X</button>
    <section id="search-results"></section>
```

```
</section>
</aside>
```

3. Scripts necesarios:

```
<!-- i18next para internacionalización -->
<script src="https://cdn.jsdelivr.net/npm/i18next@23.7.6/i18next.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/i18next-browser-languagedetector"></script>

<!-- Configuración y scripts del sitio -->
<script src="js/i18n-config.js"></script>
<script src="js/pages.js"></script>
<script src="js/search-auto.js"></script>
```

Configuración de Páginas a Indexar

En `js/pages.js`:

```
const pagesToIndex = [
  {
    url: "index.html",
    nameKey: "nav.inicio",      // Clave de traducción para el título
    metaKey: "meta.index"     // Clave de traducción para la descripción
  },
  {
    url: "about.html",
    nameKey: "nav.sobre-mi",
    metaKey: "meta.about"
  },
  // ... más páginas
];
```

Estilos CSS Necesarios

Los estilos principales están en `css/style.css`:

- `.search-overlay`: Overlay de fondo oscuro
 - `.search-results-container`: Contenedor blanco con resultados
 - `.search-result-item`: Cada resultado individual
 - `.search-snippet mark`: Resaltado de términos encontrados
 - `.search-no-results`: Mensaje de sin resultados
-

Internacionalización

Configuración de Traducciones

En `js/i18n-config.js` se definen las traducciones:

```
const TRANSLATION_RESOURCES = {
  es: {
    translation: {
      'search.placeholder': 'Buscar en el sitio...',
      'search.tooltip': 'Buscar contenido en el sitio',
      'search.no-results': 'No se encontraron resultados para:',
      'search.results': 'Resultados de búsqueda para:',
      'search.result': 'resultado',
      'search.results-plural': 'resultados'
    }
  },
  en: {
    translation: {
      'search.placeholder': 'Search the site...',
      'search.tooltip': 'Search site content',
      'search.no-results': 'No results found for:',
      'search.results': 'Search results for:',
      'search.result': 'result',
      'search.results-plural': 'results'
    }
  }
};
```

Elementos Traducibles

Los elementos con `data-i18n` se traducen automáticamente:

```
<input
  type="search"
  id="search-input"
  data-i18n="search.placeholder"
  placeholder="Buscar en el sitio...">
```

Durante la indexación, el contenido se traduce al idioma actual antes de añadirse al índice, lo que permite búsquedas en el idioma seleccionado.

Rendimiento y Optimizaciones

Optimizaciones Implementadas

1. **Debounce:** Evita búsquedas excesivas durante la escritura
2. **Indexación única:** Las páginas se indexan una sola vez al cargar
3. **Re-indexación inteligente:** Solo se re-indexa al cambiar idioma
4. **Búsqueda en memoria:** Todo el índice se mantiene en RAM para búsquedas instantáneas
5. **Eliminación de contenido irrelevante:** Se excluyen navegación, footer, scripts

Consideraciones de Rendimiento

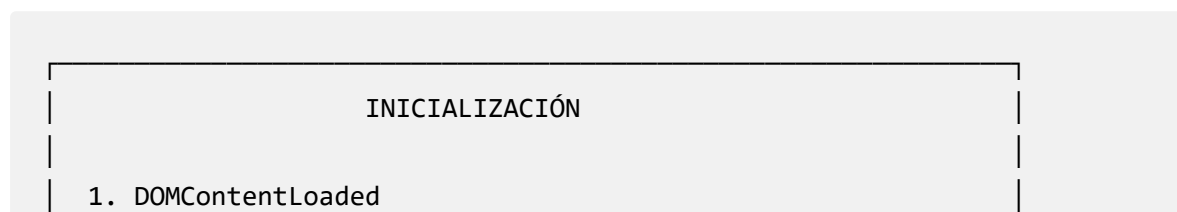
- **Tiempo de indexación:** ~100-500ms dependiendo del número de páginas
- **Tiempo de búsqueda:** <10ms para consultas típicas
- **Memoria utilizada:** ~50-200KB dependiendo del contenido
- **Ancho de banda:** Se descarga el HTML de cada página una vez

Accesibilidad

Características de Accesibilidad

1. **Roles ARIA:**
 - `role="search"` en el formulario
 - `role="dialog"` y `aria-modal="true"` en el overlay
2. **Labels descriptivos:**
 - `aria-label` en todos los controles interactivos
3. **Navegación por teclado:**
 - Tecla Escape cierra el overlay
 - Formulario enviable con Enter
4. **Foco gestionado:**
 - El overlay bloquea el scroll del body
 - El botón de cerrar es fácilmente accesible
5. **Contraste:** El resaltado con `<mark>` usa color amarillo (#ffeb3b) con buen contraste

Diagrama de Flujo Completo



```

2. Esperar i18next initialized
3. initializeSearchEngine()
  └─ new AutoSearchEngine()
  └─ initializeUI() (configurar eventos)
  └─ indexPages()
    └─ Para cada página en pagesToIndex:
      └─ fetch(url)
      └─ HTMLParser.parse()
        └─ Crear contenedor DOM
        └─ Eliminar elementos excluidos
          └─ ContentTranslator.translate()
        └─ ContentExtractor.extractMain()
          └─ Encontrar <main>
          └─ Eliminar secciones excluidas
            └─ Convertir headings a texto
          └─ Retornar objeto indexado
      └─ SearchEngine.loadIndex(pages)

```



USUARIO BUSCA

```

1. Usuario escribe en input
2. SearchEventManager.debounceSearch()
  └─ Esperar 300ms sin nuevas teclas
3. SearchEventManager.executeSearch(query)
4. AutoSearchEngine.search(query)
  └─ SearchEngine.search(term)
    └─ Para cada página en index:
      └─ calculateRelevance(page, term)
      └─ Ordenar por relevancia descendente
    └─ Para cada resultado:
      └─ SnippetBuilder.buildSnippet(page, term)
        └─ selectBestText() (donde está el término)
        └─ TextProcessor.extractContext()
          └─ findSentenceStart()
          └─ findSentenceEnd()
          └─ adjustToWorldBoundary()
          └─ removeRedundantPrefixes()
        └─ TextProcessor.highlight() (marcar término)
5. AutoSearchEngine.displayResults(results, query)
6. UIRenderer.displayResults()
  └─ TemplateRenderer.renderResults()

```

```
| └─ renderItem() para cada resultado  
| └─ Mostrar overlay  
|
```



CAMBIO DE IDIOMA

- ```
| 1. Usuario cambia selector de idioma
| 2. i18next.changeLanguage(newLang)
| 3. Evento 'languageChanged' se dispara
| 4. handleLanguageChange(lng)
| └─ Actualizar contenido de la página
| └─ autoSearchEngine.indexPages(pagesToIndex, true)
| └─ Repetir proceso de indexación con nuevo idioma
|
```

## Conclusión

Este buscador proporciona una solución completa de búsqueda client-side con soporte multiidioma, búsqueda ponderada inteligente y snippets contextuales. Su arquitectura modular facilita el mantenimiento y la extensión según las necesidades del sitio web.